

# From Questionnaire to Dataset

120 minutes · 6 self-check questions

Jesse Anak · Institute of Computational Biology and Bioinformatics

*Turning completed forms into a rectangular dataset a computer can read: the codebook, variable naming, data entry, and the decisions that are expensive to reverse later.*

## Learning objectives

- Build a codebook that documents every variable and its coding
- Name and type variables so the dataset survives being handed to someone else
- Choose between wide and long layout for your design
- Set up double entry or electronic capture to control transcription error
- Import a dataset into SPSS and R and confirm it arrived intact

## The codebook

- One row per variable: name, label, type, coding, permitted range, and the question it came from
- Write it before data entry begins, not afterwards — it is the specification, not the documentation
- Record the exact wording of the question, because "income" means nothing six months later
- Note the unit of measurement: kilograms or pounds, cedis or dollars, days or weeks
- Every recode you invent later gets added to the codebook, so the file always describes the data you actually have

### IN PRACTICE

*The codebook is what lets someone else — a supervisor, a co-author, a reviewer, or you in a year — understand the dataset without asking you. A study without one is a study only its author can analyse.*

## Structuring the dataset

- One row per unit of analysis, one column per variable, one value per cell
- The unit of analysis is usually the participant, but may be the visit, the household or the sample
- Wide format: one row per participant, repeated measures in separate columns (bp\_baseline, bp\_6months)
- Long format: one row per participant per occasion, with a time variable — required by most mixed-model and repeated-measures procedures

- Never store two pieces of information in one column ("35M" for age and sex)
- Never use colour or bold to encode meaning; a computer cannot read formatting

## Naming and typing variables

- Short, lowercase, no spaces, no accents: age\_years, sex, bp\_systolic
- Do not start a name with a digit, and avoid names that are reserved words in R or SPSS
- Prefix grouped items consistently: phq\_1 to phq\_9, so they can be selected as a block
- Types: numeric for measurements, integer for counts, factor or categorical for groups, date for dates
- Coding categories as numbers with value labels is standard in SPSS; in R prefer factors with meaningful levels
- Reserve a consistent code for missing — and never use 0, 9 or 99 if those are plausible real values

### A CODEBOOK EXTRACT

| Variable    | Label              | Type        | Coding / range                                        |
|-------------|--------------------|-------------|-------------------------------------------------------|
| id          | Participant ID     | integer     | 1-600, unique                                         |
| sex         | Sex                | categorical | 1 = male, 2 = female                                  |
| age_years   | Age at interview   | numeric     | 18-95                                                 |
| edu         | Highest education  | ordinal     | 1 = none, 2 = primary, 3 = JHS, 4 = SHS, 5 = tertiary |
| bp_systolic | Systolic BP (mmHg) | numeric     | 70-260                                                |
| nhis        | Has NHIS cover     | binary      | 0 = no, 1 = yes                                       |
| —           | Missing value code | —           | -99 across all variables                              |

## Getting the data in

- Electronic capture (KoBoToolbox, REDCap, ODK) enforces ranges and skip logic at the point of entry and removes transcription entirely
- On paper, use double entry: two people enter independently, then compare and reconcile differences
- Single entry with a 10% verification sample is the minimum defensible alternative
- Enter data exactly as recorded; correct implausible values as a documented cleaning step, not silently during entry
- Keep the raw file untouched and read-only — all cleaning happens in a script or a documented copy
- Back up before you start: three copies, two media, one off-site

---

## Reading it into your software

- CSV is the safest interchange format; Excel silently reformats dates and long ID numbers
- In R: `dat <- read.csv("survey.csv", na.strings = "-99")`
- In SPSS: File > Import Data > CSV, then set measurement level and missing values in Variable View
- Immediately check dimensions, variable names and types against the codebook
- `str(dat)` in R, or Variable View in SPSS, will show a numeric variable that arrived as text
- A variable that should be numeric but imported as character almost always means a stray value — "N/A", a comment, or a thousands separator

---

## Worked examples

### The spreadsheet that cannot be analysed

*A student arrives with 340 completed questionnaires typed into Excel. Ages are recorded as "35 yrs", "about 40" and "1987". Sex is coded M, F, m, f and Male. Missing answers are blank in some columns and "N/A" in others.*

1. Nothing can be computed yet: age is text, so R will not take a mean and SPSS will treat it as nominal.
2. "1987" is a birth year, not an age — two different variables were entered into one column.
3. Five spellings of two categories means `table(sex)` returns five groups instead of two.
4. Blank and "N/A" both mean missing, but only blank will be recognised on import.
5. The fix is not cleverer code, it is a codebook and re-entry: age\_years numeric, sex coded 1/2, one missing code throughout.
6. Re-entering 340 records takes two days. Untangling this by script takes longer and leaves doubt about every value.

---

### THE POINT

Data entry decisions are analysis decisions. An hour spent on the codebook before entry saves days afterwards, and the days afterwards never fully restore confidence in the data.

### Wide or long: the same data, two shapes

*Blood pressure measured at baseline, 3 and 6 months for 120 patients. The student wants a repeated-measures analysis and the procedure refuses to run.*

1. The data are wide: one row per patient with bp\_o, bp\_3, bp\_6 as three columns.
2. Wide is convenient for computing change scores: `dat$change <- dat$bp_6 - dat$bp_o`.
3. Mixed models and most repeated-measures procedures need long: one row per patient per occasion.
4. In R: `long <- reshape(dat, direction = "long", varying = c("bp_o", "bp_3", "bp_6"), v.names = "bp", timevar = "month")`
5. Or with tidyr: `pivot_longer(dat, cols = starts_with("bp_"), names_to = "month", values_to = "bp")`
6. Keep both. They are the same data, and converting between them is two lines.

---

#### THE POINT

Neither shape is correct in general — the analysis decides. Knowing how to convert takes the argument away entirely.

---

## Key terms

### Codebook

A document defining every variable, its type, coding and permitted values.

### Unit of analysis

The entity each row of the dataset represents — participant, visit, household.

### Wide format

One row per participant, with repeated measures held in separate columns.

### Long format

One row per participant per occasion, with an explicit time variable.

### Double entry

Entering paper data twice independently and reconciling differences to catch transcription error.

### Missing value code

A reserved value marking absent data, chosen so it cannot be confused with a real observation.

---

## Activity

Take one page of your own questionnaire and write the codebook rows for every variable on it: name, label, type, coding and permitted range. Swap with a partner and try to enter three fictional respondents using only their codebook. Every question you have to ask them is a gap in it.

---

## Further reading

- **Broman & Woo (2018)** — "Data Organization in Spreadsheets" — Short, free, and the best thing written on this. Read it before you enter any data.
- **Wickham (2014)** — "Tidy Data" — Why one variable per column matters, with examples of what goes wrong.
- **REDCap and KoBoToolbox documentation** — Both free for academic use; either removes a whole class of entry error.

---

## Self-check questions

1. A column in your dataset contains entries like "35M" and "42F". The problem is that it:
  - a. Uses the wrong missing value code
  - b. Holds two variables in one column, so neither can be analysed
  - c. Is in long format
  - d. Should have been entered in Excel
2. Why is 99 a poor choice of missing value code for a variable recording age?
  - a. SPSS cannot handle two-digit codes
  - b. 99 is a plausible real age, so genuine and missing values become indistinguishable
  - c. It must always be negative
  - d. It conflicts with the participant ID
3. You measure blood pressure at baseline, 3 months and 6 months, and plan a repeated-measures analysis. The dataset should be in:
  - a. Wide format only
  - b. Long format, one row per participant per occasion
  - c. Whatever format it was entered in
  - d. Neither — repeated measures cannot be stored in a rectangle
4. A variable you know is numeric imports into R as character. The most likely cause is:

- a. The file was too large
  - b. A stray non-numeric entry somewhere in the column, such as "N/A" or a comment
  - c. R cannot read CSV files
  - d. The variable name starts with a letter
5. The strongest reason to keep the raw data file untouched is that:
- a. It saves disk space
  - b. Every cleaning decision stays reversible and auditable, because the original still exists
  - c. SPSS requires it
  - d. It makes the analysis run faster
6. When should the codebook be written?
- a. After analysis, as documentation
  - b. Before data entry begins, because it is the specification for the dataset
  - c. Only if the study is published
  - d. At the same time as the discussion chapter
- 

## Answers

- 1. **b.** Age and sex are separate variables and need separate columns. As stored, you cannot compute a mean age or cross-tabulate by sex without first splitting the column — work that should never have been necessary.
  - 2. **b.** A missing code must be impossible as a real observation. For age, 99 is attainable; -99 is not. The same logic rules out 0 for variables where zero is a genuine value.
  - 3. **b.** Repeated-measures and mixed-model procedures expect long format: a row per observation with an explicit time variable. Wide is convenient for entry and for computing change scores, and converting between them is routine.
  - 4. **b.** A single non-numeric value forces the whole column to character. Find it with something like `unique(dat$var[is.na(as.numeric(dat$var))])` — it is usually a typed note, a unit, or a missing marker the import did not know about.
  - 5. **b.** Cleaning in place destroys the record of what the data originally said. Keeping the raw file read-only and doing all changes in a script means any decision can be revisited, and a reviewer can see exactly what you altered.
  - 6. **b.** The codebook defines what will be entered and how. Written afterwards it is archaeology, and it will not match the file, because entry decisions were made ad hoc while typing.
-